

Lunes 2 de marzo de 2026

¡Hola José! No hice todo lo que hablamos la semana pasada, pero pienso que lo que hice da para vernos.

Índice

1. Sobre la semana pasada	1
1.1. <i>Weight sharing</i> del diccionario	1
1.2. Mapas de activación	2
1.3. Me queda pendiente	2
2. Sobre el coste de la atención	2
3. Necesidad de otra tarea	3
4. La configuracion de todos estos experimentos	3
5. ¿Qué creo que tengo que hacer?	4
6. Extra...	4

1. Sobre la semana pasada

Implementé el *weight sharing* de los diccionarios de ISTA y el truco para tener más resolución en los mapas de activación.

1.1. *Weight sharing* del diccionario

Lo cierto es que los entrenamientos son mucho más rápidos, los modelos más pequeños y [comentar diferencias].

1.2. Mapas de activación

Ahora que son más resolutivos vemos que: las primeras cabezas implementan derivadas primeras de x e y y las atenciones responden a información de ese token y ya, pero en capas más profundas responden a la imagen como un todo (por eso vemos que los tokens de las cabezas finales son más o menos parecidos entre sí, porque la información que portan ya no tiene tanto que ver con el parche de la imagen sino con la tarea (*¿hay vena en el píxel central de la imagen?*)).

1.3. Me queda pendiente

- Mezclar órdenes inferencia / entrenamiento.
- Desacoplar *positional embeddings*.

2. Sobre el coste de la atención

Esta sección discute una posibilidad que no implementé. Es decir, más elucubración, pero bueno, espero que no te moleste.

La semana pasada me recomendaste la lectura de DeepSeek por lo de la mezcla de expertos. Me lo imprimí. Como algo ya conocía de eso por Técnicas Avanzadas del Procesamiento del Lenguaje Natural, y sabía más o menos cómo era el truco, no me motivaba mucho... Busqué y encontré un artículo estupendo: el Linformer <https://arxiv.org/pdf/2006.04768>. Su truco es:

Imagínate una atención genérica $\text{softmax}(Q'K/\sqrt{d})V$. Tú me venías alertando de la complejidad $\mathcal{O}(n^2)$ porque Q' es $n \times d$; entonces lo que va dentro del softmax es $n \times n$ y luego si multiplicas por V ahí está el tema. Evidentemente tú y yo sabemos de primero de álgebra que lo que va dentro del softmax no puede ser de rango mayor que d . Esencialmente el artículo demuestra que el softmax no cambia eso. ¿A dónde quiero llegar? Si sabemos que estamos haciendo un producto de matrices grandes de rango bajo, de buena manera podemos hacer un producto de matrices pequeñas de rango completo sin perder información; de forma natural:

$$\text{softmax}(Q'KE/\sqrt{d})FV$$

donde E y F son las matrices de reducción y ampliación de dimensionalidad respectivamente.

3. Necesidad de otra tarea

Lo cierto es que esta tarea es muy sencilla. Yo creo que fue apropiada para ir haciendo pruebas y a mí me valió para tener una referencia clara de qué debería hacer el modelo, pero como es una tarea muy simple, no hacen falta ni muchos tokens de entrada ni redes muy grandes, por tanto las cuestiones de rendimiento más o menos se eluden.

Imagina el caso de que implemento el Linformer, y paso de $\mathcal{O}(n^2)$ a $\mathcal{O}(nr)$ donde r es el rango. Pues si n es 5 ó 7 y r es (pongamos) 3 ó 2, hasta seguramente veamos peores rendimientos por el hecho de introducir una matriz más.

Además, pienso que los mapas de activación serán más interesantes porque si pensamos los mapas de activación como el “razonamiento” de la red, pues (pienso yo) para una tarea trivial el razonamiento es trivial.

Entonces básicamente te estoy preguntando: primero, ¿estás de acuerdo con lo que acabo de decir? Y segundo, ¿qué dataset de patología podríamos escoger para ya *simular* el asistente del médico?

Lo primero que me preguntaste el primer día era si quería hacer un TFG de investigación o de aplicación, y desde luego me lo pasé genial con la elección. Sigo sin *querer* un TFG de aplicación, pero pienso que la aplicación motivaría los problemas que la investigación resuelve.

4. La configuracion de todos estos experimentos

```
arch: CRATE_enana2nd_shared_dict
workers: 16
epochs: 5000
label_smoothing: 0.0
batch_size: 4096
lr: 5.0e-05
momentum: 0.9
use_amp: false
weight_decay: 0.001
aumento_datos: true
seed: null
tamano_patch: 75
```

```
tamano_token: 15
contador_aumento: -1
overlap_rate: 0.5
label_mode: vainilla
optimizer: AdamW
paciencia: 1000
warmup_steps: 20
class_weight: 1.0
```

5. ¿Qué creo que tengo que hacer?

¿Parar de traer cosas al frente y hacer las cosas que fui dejando? ¿Tirar con algo de lo que te conte ahora? Pues no lo sé.

6. Extra...

¡Fui de ayuda a alguien! :D

Out Projection Matrix Different than Subspace Matrix #31

New issue

Closed

Carlson-J opened last week

```
CRATE/model/crate.py
Line 61 in 674488f
61     nn.Linear(inner_dim, dim),
```

In the paper, equation 13, the output from SSA is projected through the U matrix, which is the same as the projections done to make the KVQ matrices. In the code, instead of using the same Linear layer's weights (same U but transposed) for the projection a new linear layer is used. This deviates from the derivation.

Is this a bug or is there some justification for not following the paper's derivation in this case, or I am not following something here?

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

Notifications Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

PepeRoConde 3 days ago

in page 24 (<https://arxiv.org/pdf/2311.13110>) them say

$$\text{MSSA}(\mathbf{Z} \mid \mathbf{U}_{[K]}, \mathbf{W}) \doteq \mathbf{W} \begin{bmatrix} \text{SSA}(\mathbf{Z} \mid \mathbf{U}_1) \\ \vdots \\ \text{SSA}(\mathbf{Z} \mid \mathbf{U}_K) \end{bmatrix}. \quad (37)$$

is this the issue you addressed?

Development

Code with agent mode

Notifications Customize

Unsubscribe

You're receiving notifications because you're subscribed to this thread.

Participants

Carlson-J closed this as completed 13 hours ago